

EXPERIMENT 21

Title: Write a Prolog Program to Implement Towers of Hanoi

Aim

To develop a Prolog program to implement the Towers of Hanoi problem using recursion and generate the sequence of moves required to transfer all the disks from the source peg to the destination peg.

Objective

- To understand the Towers of Hanoi problem.
- To implement recursion using Prolog.
- To understand the concept of divide and conquer.
- To generate the sequence of valid disk movements.
- To understand the application of recursion in Artificial Intelligence.

Theory

The Towers of Hanoi is a classical problem used to demonstrate recursion and problem-solving techniques. The problem consists of three pegs and a number of disks of different sizes.

The objective is to move all the disks from the **source peg** to the **destination peg** using an auxiliary peg.

The following rules must be satisfied:

- Only one disk can be moved at a time.
- Only the top disk of a peg can be moved.
- A larger disk cannot be placed on top of a smaller disk.
- All disks must be moved from the source peg to the destination peg.

For N disks, the minimum number of moves required is:

$$2^n - 1$$

Algorithm

Step 1: Start the program.

Step 2: Define the base case for one disk.

Step 3: For N disks, move N-1 disks from the source peg to the auxiliary peg.

Step 4: Move the largest disk from the source peg to the destination peg.

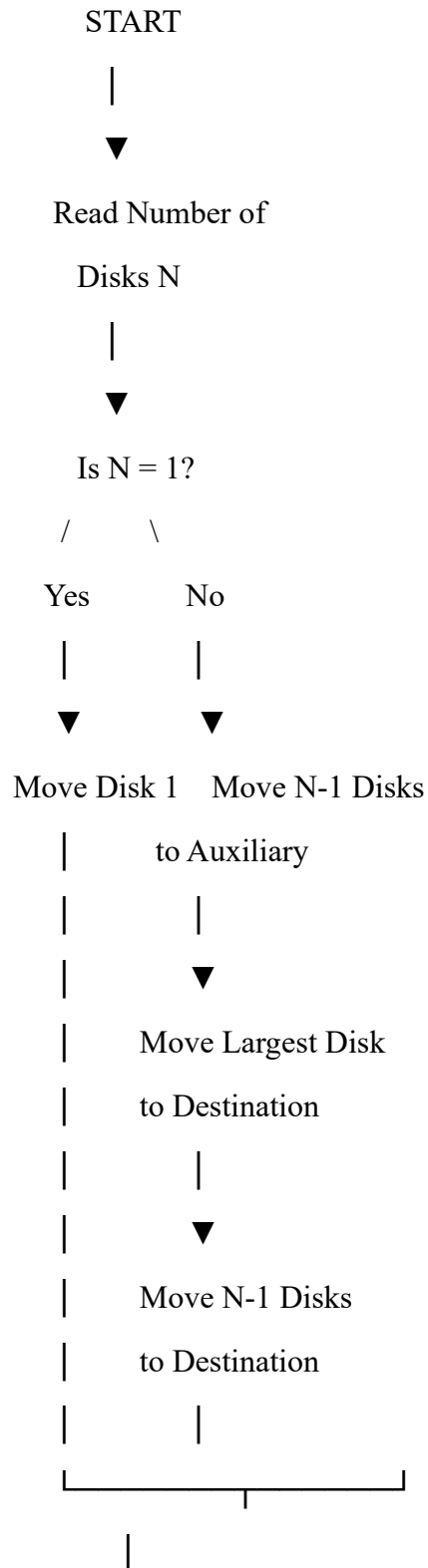
Step 5: Move the N-1 disks from the auxiliary peg to the destination peg.

Step 6: Repeat the process recursively until all disks are moved.

Step 7: Display all the moves.

Step 8: Stop the program.

Flowchart





Display Moves



STOP

Prolog Program

% Towers of Hanoi

hanoi(1, Source, Destination, _) :-

write('Move disk from '),

write(Source),

write(' to '),

write(Destination),

nl.

hanoi(N, Source, Destination, Auxiliary) :-

N > 1,

N1 is N - 1,

hanoi(N1, Source, Auxiliary, Destination),

write('Move disk from '),

write(Source),

write(' to '),

write(Destination),

nl,

hanoi(N1, Auxiliary, Destination, Source).

Query

For 3 disks:

?- hanoi(3, left, right, middle).

Sample Output

Move disk from left to right

Move disk from left to middle

Move disk from right to middle

Move disk from left to right

Move disk from middle to left

Move disk from middle to right

Move disk from left to right

true.

Explanation

The predicate:

hanoi(N, Source, Destination, Auxiliary)

represents the Towers of Hanoi problem with:

- N – Number of disks.
- Source – Starting peg.
- Destination – Final peg.
- Auxiliary – Temporary peg.

For three disks, the program follows these steps:

1. Move two disks from the left peg to the middle peg.
2. Move the largest disk from the left peg to the right peg.
3. Move the two disks from the middle peg to the right peg.

The program uses recursion to divide the problem into smaller subproblems.

For 3 disks, the minimum number of moves is:

$2^3 - 1 = 7$ moves

Advantages

- Simple implementation using recursion.
- Produces the minimum number of moves.
- Demonstrates divide-and-conquer problem solving.
- Helps understand recursive reasoning in Prolog.
- Useful for understanding state-space problems.

Applications

- Artificial Intelligence problem solving.
- Recursive algorithm design.
- Mathematical puzzle solving.
- Algorithm analysis.
- State-space search.
- Teaching recursion and logical programming.

Result

The Prolog program was successfully implemented to solve the Towers of Hanoi problem using recursion. For three disks, the program generated the minimum required sequence of **7 moves**.

Conclusion

Thus, the Towers of Hanoi problem was successfully implemented using Prolog. The experiment demonstrates how recursion can be used to divide a complex problem into smaller subproblems and generate an optimal sequence of moves.

```
Enter the number of disks: 1
```

```
Steps to solve Towers of Hanoi:  
Move disk 1 from A to C
```

```
Total number of moves: 1
```

```
1  # TOWERS OF HANOI
2
3  def tower_of_hanoi(n, source, auxiliary, destination):
4      # Base case
5      if n == 1:
6          print("Move disk 1 from", source, "to", destination)
7          return
8
9      # Move n-1 disks from source to auxiliary
10     tower_of_hanoi(n - 1, source, destination, auxiliary)
11
12     # Move the largest disk from source to destination
13     print("Move disk", n, "from", source, "to", destination)
14
15     # Move n-1 disks from auxiliary to destination
16     tower_of_hanoi(n - 1, auxiliary, source, destination)
17
18
19 # Main program
20 n = int(input("Enter the number of disks: "))
21
22 print("\nSteps to solve Towers of Hanoi:")
23 tower_of_hanoi(n, "A", "B", "C")
24
25 print("\nTotal number of moves:", (2 ** n) - 1)
```